

1.

```
import ast
import io
import tokenize
```

```
def line_number(input_filename: str, output_filename: str) -> None: """Write a copy of a
text file with each line prefixed by its line number.""" try: with open(input_filename, "r",
encoding="utf-8") as infile: with open(output_filename, "w", encoding="utf-8") as outfile:
for number, line in enumerate(infile, start=1): outfile.write(f"{number}. {line}") except
Exception as error: print(f"Error processing the file: {error}") raise
```

```
def remove_comments(code: str) -> str: """Return Python code with comment tokens
removed.""" tokens = tokenize.generate_tokens(io.StringIO(code).readline) kept_tokens =
[ token for token in tokens if token.type != tokenize.COMMENT ] return
tokenize.untokenize(kept_tokens)
```

```
def parse_functions(filename: str) -> tuple: """Parse a Python file and return information
about its functions.""" try: with open(filename, "r", encoding="utf-8") as file: source =
file.read()
```

```
    tree = ast.parse(source)
    lines = source.splitlines()
    result = []
```

```
    for node in tree.body:
        if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
            arguments = ast.unparse(node.args)
```

```
            code = "\n".join(
                lines[node.lineno - 1:node.end_lineno]
            )
```

```
            code = remove_comments(code)
```

```
            code = "\n".join(
                line.rstrip()
                for line in code.splitlines()
                if line.strip()
            ) + "\n"
```

```
            result.append(
```

```

        (node.lineno, node.name, arguments, code)
    )

    result.sort(key=lambda item: item[1])
    return tuple(result)

except Exception as error:
    print(f"Error parsing the Python file: {error}")
    raise

def main() -> None: """Test line_number and parse_functions using this source file."""
    input_filename = file output_filename = file + ".txt"

    line_number(input_filename, output_filename)

    print("Numbered file created:")
    print(output_filename)

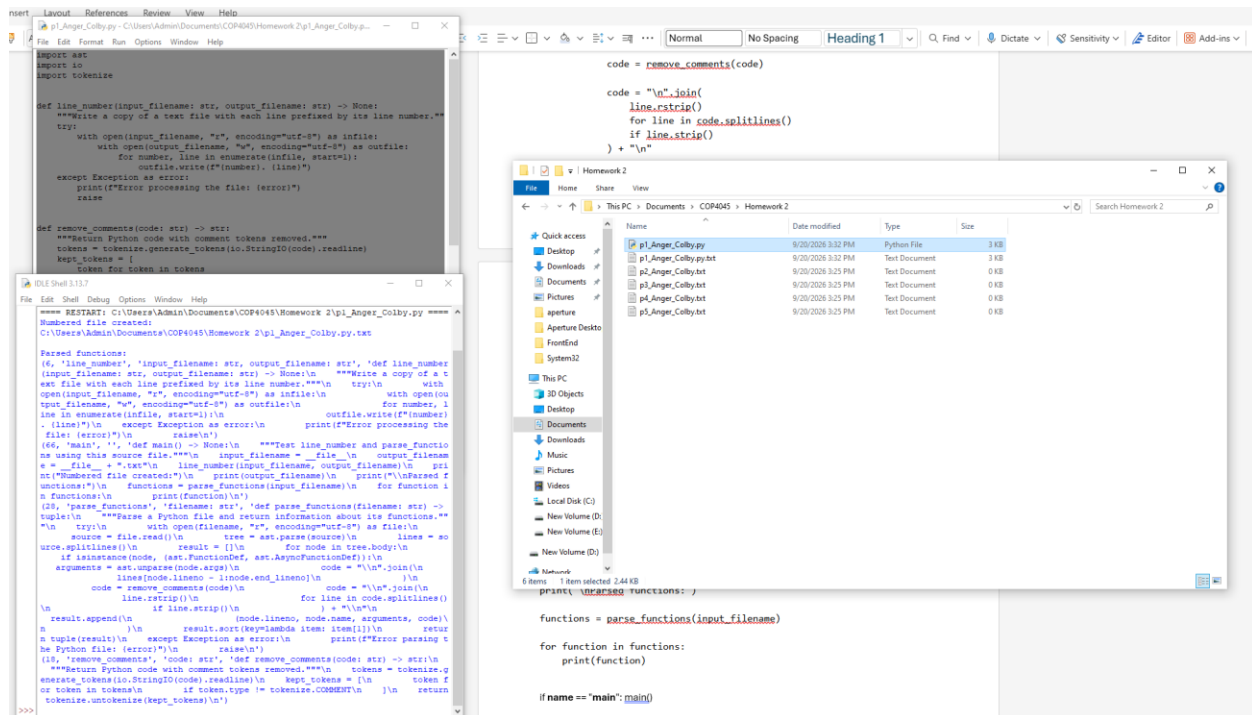
    print("\nParsed functions:")

    functions = parse_functions(input_filename)

    for function in functions:
        print(function)

if name == "main": main()

```



2.

```
def main() -> None: part_a = [(a, b, c, d) for a in range(1, 11) for b in range(1, 11) for c in
range(1, 11) for d in range(1, 11) if len({a, b, c, d}) == 4 and a ** 2 + b ** 2 == c ** 2 + d ** 2 ]
```

```
print("Part A:")
print(part_a)
```

```
strings = ["One", "SEVEN", "three", "two", "Ten"]
```

```
part_b = [
    (word.lower(), len(word))
    for word in strings
    if len(word) < 5
]
```

```
print("\nPart B:")
print(part_b)
```

```
names = [
    "Christopher Ashton Kutcher",
    "Elizabeth Stamatina Fey"
]
```

```

part_c = [
    f"{parts[0]} {parts[1][0]}. {parts[2]}"
    for parts in (name.split() for name in names)
]

print("\nPart C:")
print(part_c)

lst1 = ["Spam", "Trams", "Elbows", "Tops", "Astral"]
lst2 = ["Bowels", "Sample", "Altars", "Stop", "Course", "Smart"]

part_d = [
    (w1, w2)
    for w1 in lst1
    for w2 in lst2
    if sorted(w1.lower()) == sorted(w2.lower())
]

print("\nPart D:")
print(part_d)

s = ["one", "two", "three"]

part_e = {
    word: len(word)
    for word in s
}

print("\nPart E:")
print(part_e)

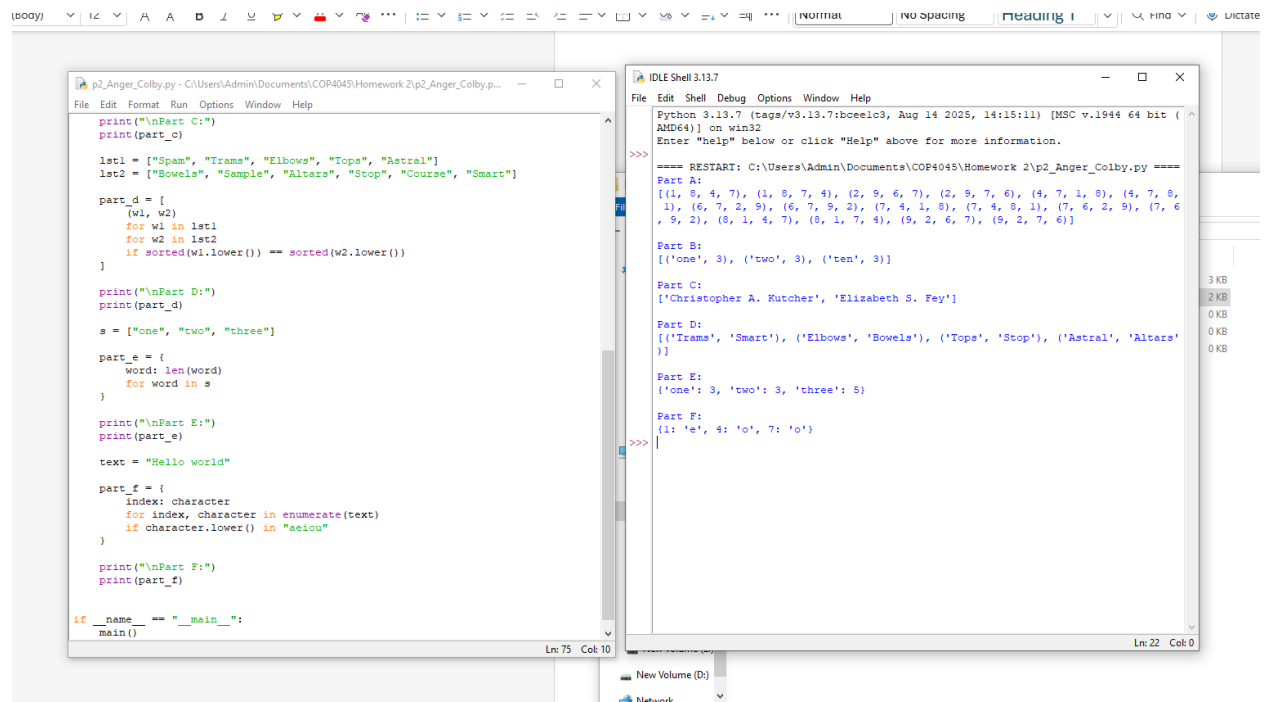
text = "Hello world"

part_f = {
    index: character
    for index, character in enumerate(text)
    if character.lower() in "aeiou"
}

```

```
print("\nPart F:")
print(part_f)
```

```
if name == "main": main()
```



3.

```
import csv
```

```
def add_user( sn: dict[str, tuple[str, list[str]]], username: str, fullname: str ) -> bool: """Add
a new user to the social network.""" try: if username in sn: return False
```

```
    sn[username] = (fullname, [])
    return True
```

```
except Exception as error:
    print(f"Error adding user: {error}")
    raise
```

```
def add_friend( sn: dict[str, tuple[str, list[str]]], user1: str, user2: str ) -> bool: """Add a mutual friendship between two users.""" try: if user1 not in sn or user2 not in sn: return False
```

```
    if user1 == user2:
        return False
```

```
    if user2 not in sn[user1][1]:
        sn[user1][1].append(user2)
```

```
    if user1 not in sn[user2][1]:
        sn[user2][1].append(user1)
```

```
    return True
```

```
except Exception as error:
    print(f"Error adding friendship: {error}")
    raise
```

```
def get_friends( sn: dict[str, tuple[str, list[str]]], user1: str, distance: int ) -> list[str]:
    """Return all friends within the specified link distance.""" try: if user1 not in sn or distance
    <= 0: return []
```

```
    visited = {user1}
    current_level = [user1]
    result = []
```

```
    for _ in range(distance):
        next_level = []
```

```
        for user in current_level:
            for friend in sn[user][1]:
                if friend not in visited:
                    visited.add(friend)
                    result.append(friend)
                    next_level.append(friend)
```

```
        current_level = next_level
```

```

        if not current_level:
            break

    return result

except Exception as error:
    print(f"Error getting friends: {error}")
    raise

def save_network( filename: str, sn: dict[str, tuple[str, list[str]]] ) -> None: """Save a social
network dictionary to a CSV file.""" try: with open(filename, "w", newline="", encoding="utf-
8") as file: writer = csv.writer(file)

        for username, data in sn.items():
            fullname, friends = data
            writer.writerow([username, fullname] + friends)

except Exception as error:
    print(f"Error saving network: {error}")
    raise

def load_network( filename: str ) -> dict[str, tuple[str, list[str]]]: """Load a social network
dictionary from a CSV file.""" try: sn = {}

    with open(filename, "r", newline="", encoding="utf-8") as file:
        reader = csv.reader(file)

        for row in reader:
            if len(row) >= 2:
                username = row[0]
                fullname = row[1]
                friends = row[2:]
                sn[username] = (fullname, friends)

    return sn

except Exception as error:

```

```

    print(f"Error loading network: {error}")
    raise

def main() -> None: """Test the social network functions.""" sn = { "alice": ("Alice Smith",
["maria"]), "maria": ("Maria Cortez", ["alice", "joe", "david"]), "joe": ("Joseph Adams",
["maria", "eve"]), "eve": ("Evelyn Cooper", ["joe"]), "david": ("David Benson", ["maria"]) }

print("Original network:")
print(sn)

print("\nAdd user:")
print(add_user(sn, "bob", "Bob Johnson"))

print("\nAdd friendship:")
print(add_friend(sn, "bob", "alice"))

print("\nAlice - distance 1:")
print(get_friends(sn, "alice", 1))

print("\nAlice - distance 2:")
print(get_friends(sn, "alice", 2))

print("\nAlice - distance 3:")
print(get_friends(sn, "alice", 3))

filename = "social_network.csv"

save_network(filename, sn)

print("\nLoaded network:")
print(load_network(filename))

if name == "main": main()

```

```

def main() -> None:
    """Test the social network functions."""
    sn = {
        "alice": ("Alice Smith", ["maria"]),
        "maria": ("Maria Cortez", ["alice", "joe", "david"]),
        "joe": ("Joseph Adams", ["maria", "eve"]),
        "eve": ("Evelyn Cooper", ["joe"]),
        "david": ("David Benson", ["maria"])
    }

    print("Original network:")
    print(sn)

    print("\nAdd user:")
    print(add_user(sn, "bob", "Bob Johnson"))

    print("\nAdd friendship:")
    print(add_friend(sn, "bob", "alice"))

    print("\nAlice - distance 1:")
    print(get_friends(sn, "alice", 1))

    print("\nAlice - distance 2:")
    print(get_friends(sn, "alice", 2))

    print("\nAlice - distance 3:")
    print(get_friends(sn, "alice", 3))

    filename = "social_network.csv"

    save_network(filename, sn)

    print("\nLoaded network:")
    print(load_network(filename))

if __name__ == "__main__":
    main()

```

```

Python 3.13.7 (tags/v3.13.7:bce1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
==== RESTART: C:\Users\Admin\Documents\COP4045\Homework 2\p3_Anger_Colby.py ====
Original network:
{'alice': ('Alice Smith', ['maria']), 'maria': ('Maria Cortez', ['alice', 'joe', 'david']), 'joe': ('Joseph Adams', ['maria', 'eve']), 'eve': ('Evelyn Cooper', ['joe']), 'david': ('David Benson', ['maria'])}

Add user:
True

Add friendship:
True

Alice - distance 1:
['maria', 'bob']

Alice - distance 2:
['maria', 'bob', 'joe', 'david']

Alice - distance 3:
['maria', 'bob', 'joe', 'david', 'eve']

Loaded network:
{'alice': ('Alice Smith', ['maria', 'bob']), 'maria': ('Maria Cortez', ['alice', 'joe', 'david']), 'joe': ('Joseph Adams', ['maria', 'eve']), 'eve': ('Evelyn Cooper', ['joe']), 'david': ('David Benson', ['maria']), 'bob': ('Bob Johnson', ['alice'])}
>>>

```

4.

import csv

def load_rated_movies(filename: str) -> dict: """Load the top-rated movies into a dictionary.""" movies = {}

with open(filename, "r", encoding="utf-8", newline="") as file:
 reader = csv.DictReader(file)

for row in reader:
 movies[(row["Title"], row["Year"])] = float(
 row["IMDb Rating"]
)

return movies

def load_grossing_movies(filename: str) -> dict: """Load the top-grossing movies into a dictionary.""" movies = {}

with open(filename, "r", encoding="utf-8", newline="") as file:
 reader = csv.DictReader(file)

```

    for row in reader:
        money = row["USA Box Office"]
        money = money.replace("$", "").replace(",", "").strip()

        movies[(row["Title"], row["Year"])] = float(money)

return movies


def load_casts(filename: str) -> dict: """Load movie directors and casts into a dictionary."""
movies = {}

with open(filename, "r", encoding="utf-8", newline="") as file:
    reader = csv.reader(file)

    for row in reader:
        if len(row) >= 3:
            title = row[0]
            year = row[1]
            director = row[2]

            actors = [
                actor
                for actor in row[3:8]
                if actor.strip()
            ]

            movies[(title, year)] = (director, actors)

return movies


def display_ranking(ranking: list, limit: int | None = None) -> None: """Display a ranking with
an optional maximum number of entries.""" if limit is not None: ranking = ranking[:limit]

for position, item in enumerate(ranking, start=1):
    print(position, item)

```

```

def display_top_collaborations( rated_filename: str, casts_filename: str, limit: int | None =
None ) -> None: """Display director/actor pairs ranked by number of top-rated movies."""
rated_movies = load_rated_movies(rated_filename) casts = load_casts(casts_filename)

collaborations = {}

for movie in rated_movies:
    if movie in casts:
        director, actors = casts[movie]

        for actor in actors:
            pair = (director, actor)
            collaborations[pair] = collaborations.get(pair, 0) + 1

ranking = [
    (director, actor, count)
    for (director, actor), count in collaborations.items()
]

ranking.sort(key=lambda item: item[2], reverse=True)

display_ranking(ranking, limit)


def display_top_actors( grossing_filename: str, casts_filename: str, limit: int | None =
None ) -> None: """Display actors ranked by total box-office money.""" grossing_movies =
load_grossing_movies(grossing_filename) casts = load_casts(casts_filename)

totals = {}

for movie, box_office in grossing_movies.items():
    if movie in casts:
        _, actors = casts[movie]

        for actor in actors:
            totals[actor] = totals.get(actor, 0) + box_office

ranking = list(totals.items())
ranking.sort(key=lambda item: item[1], reverse=True)

```

```
display_ranking(ranking, limit)
```

```
def main() -> None: """Display the top ten results for both rankings.""" rated_file = "imdb-  
top-rated.csv" grossing_file = "imdb-top-grossing.csv" casts_file = "imdb-top-casts.csv"
```

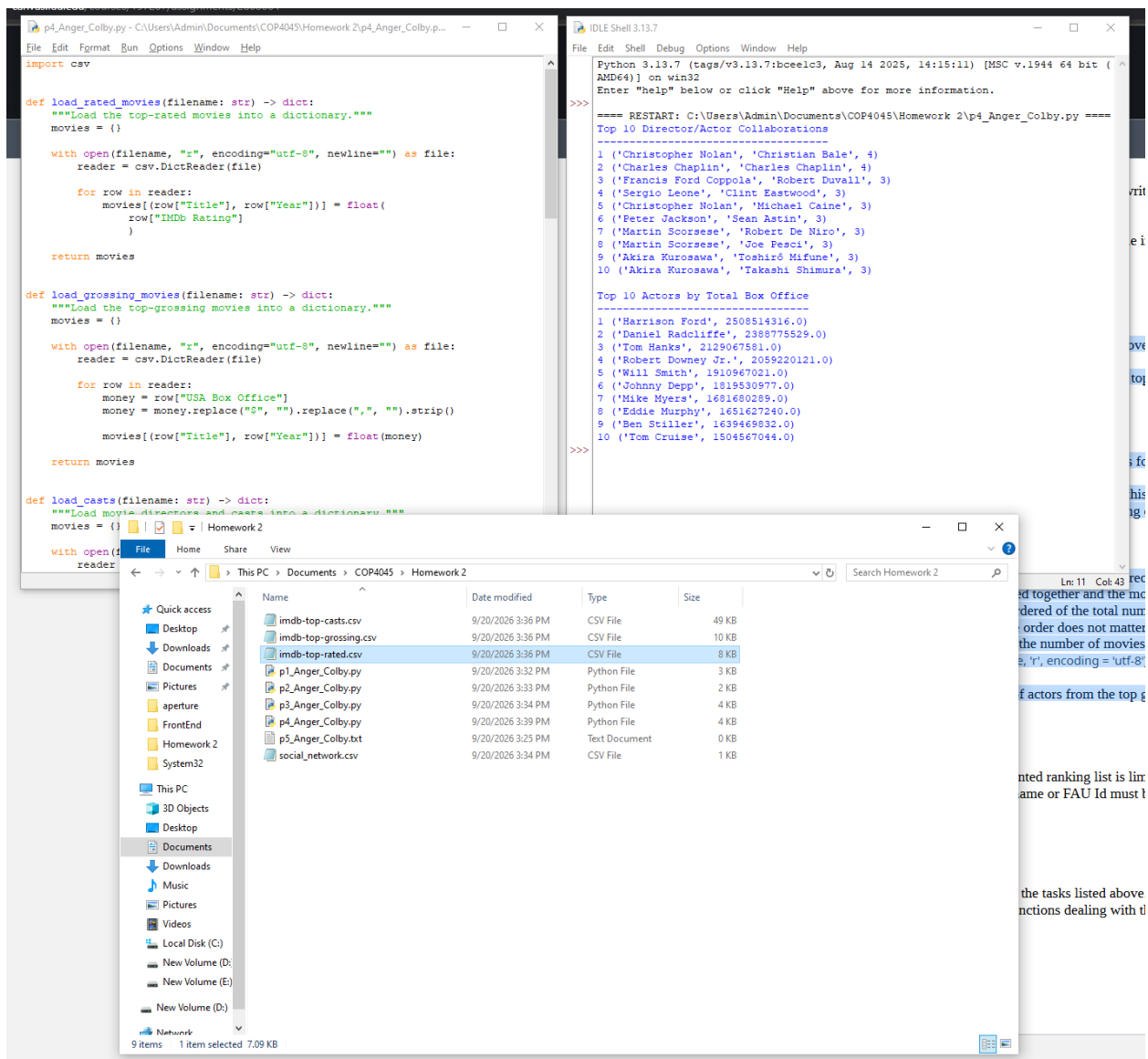
```
print("Top 10 Director/Actor Collaborations")  
print("-----")
```

```
display_top_collaborations(rated_file, casts_file, 10)
```

```
print("\nTop 10 Actors by Total Box Office")  
print("-----")
```

```
display_top_actors(grossing_file, casts_file, 10)
```

```
if name == "main": main()
```



5.

```
import sys
import unittest
from datetime import datetime
from tempfile import NamedTemporaryFile
from unittest.mock import patch

DATE_FORMAT = "%I:%M:%S %p %m/%d/%Y"
MIN_TEMPERATURE = -100.0
MAX_TEMPERATURE = 150.0
```

```

def read_observations(filename):
    """Read valid station observations and return them with line errors."""
    observations = {}
    errors = []
    seen = set()

    with open(filename, "r", encoding="utf-8") as input_file:
        for line_number, line in enumerate(input_file, start=1):
            fields = line.rstrip("\n\r").split(",")
            if len(fields) != 3:
                errors.append((line_number, "Malformed line"))
                continue

            station, date_text, temperature_text = (field.strip() for field in
fields)

            if not station or not date_text or not temperature_text:
                errors.append((line_number, "Malformed line"))
                continue

            try:
                date = datetime.strptime(date_text, DATE_FORMAT)
            except ValueError:
                errors.append((line_number, "Malformed line"))
                continue

            try:
                temperature = float(temperature_text)
            except ValueError:
                errors.append((line_number, "Invalid temperature"))
                continue

            if not MIN_TEMPERATURE <= temperature <= MAX_TEMPERATURE:
                errors.append((line_number, "Invalid temperature"))
                continue

            observation_key = (station, date)
            if observation_key in seen:
                errors.append((line_number, "Duplicate observation"))
                continue

```

```

        seen.add(observation_key)
        observations.setdefault(station, []).append((date, temperature))

    for station in observations:
        observations[station].sort(key=lambda observation: observation[0])

    return observations, errors


def station_statistics(observations):
    """Return minimum, maximum, and mean temperature for every station."""
    return {
        station: (
            min(temperature for _, temperature in station_observations),
            max(temperature for _, temperature in station_observations),
            sum(temperature for _, temperature in station_observations)
            / len(station_observations),
        )
        for station, station_observations in observations.items()
    }


def station_outliers(observations):
    """Return stations whose latest temperature is above their station mean."""
    statistics = station_statistics(observations)
    return {
        station: (latest_date, latest_temperature, statistics[station][2])
        for station, station_observations in observations.items()
        for latest_date, latest_temperature in [station_observations[-1]]
        if latest_temperature > statistics[station][2]
    }


def write_statistics(filename, statistics):
    """Write station statistics in lexicographic station order."""
    with open(filename, "w", encoding="utf-8") as output_file:
        for station in sorted(statistics):
            minimum, maximum, mean = statistics[station]
            output_file.write(
                f"{station},{minimum:.1f},{maximum:.1f},{mean:.1f}\n"
            )

```

```

def main():
    """Read the command-line files, print results, and write statistics."""
    if len(sys.argv) != 3:
        print("Usage: python p5_Anger_Colby.py input_file output_file")
        return

    input_filename, output_filename = sys.argv[1:]
    try:
        observations, errors = read_observations(input_filename)
        statistics = station_statistics(observations)
        outliers = station_outliers(observations)
        write_statistics(output_filename, statistics)
    except OSError as error:
        print(f"File access error: {error}")
        return

    print("Statistics:")
    for station in sorted(statistics):
        minimum, maximum, mean = statistics[station]
        print(f"{station}: min={minimum:.1f}, max={maximum:.1f},
mean={mean:.1f}")

    print("Outliers:")
    for station in sorted(outliers):
        date, temperature, mean = outliers[station]
        print(f"{station}: {date.strftime(DATE_FORMAT)}, {temperature:.1f},
{mean:.1f}")

    if errors:
        print("Errors:")
        for line_number, error_message in errors:
            print(f"line {line_number}: {error_message}")

class TestTemperatureAnalysis(unittest.TestCase):
    def create_input_file(self, contents):
        input_file = NamedTemporaryFile(mode="w", encoding="utf-8", delete=False)
        input_file.write(contents)
        input_file.close()
        self.addCleanup(lambda: self.remove_file(input_file.name))
        return input_file.name

```

```

@staticmethod
def remove_file(filename):
    import os

    if os.path.exists(filename):
        os.remove(filename)

def test_reads_several_stations_and_sorts_observations(self):
    filename = self.create_input_file(
        "Beta,09:28:09 AM 04/20/2026,70\n"
        "Alpha,09:28:09 AM 04/19/2026,60\n"
        "Beta,09:28:09 AM 04/19/2026,65\n"
    )
    observations, errors = read_observations(filename)
    self.assertEqual(errors, [])
    self.assertEqual(list(observations), ["Beta", "Alpha"])
    self.assertLess(observations["Beta"][0][0], observations["Beta"][1][0])

def test_accepts_negative_temperatures(self):
    filename = self.create_input_file("North,09:28:09 AM 04/20/2026,-40.5\n")
    observations, errors = read_observations(filename)
    self.assertEqual(errors, [])
    self.assertEqual(observations["North"][0][1], -40.5)

def test_rejects_duplicate_observations(self):
    filename = self.create_input_file(
        "North,09:28:09 AM 04/20/2026,40\n"
        "North,09:28:09 AM 04/20/2026,41\n"
    )
    observations, errors = read_observations(filename)
    self.assertEqual(len(observations["North"]), 1)
    self.assertEqual(errors, [(2, "Duplicate observation")])

def test_rejects_invalid_temperature_ranges(self):
    filename = self.create_input_file(
        "A,09:28:09 AM 04/20/2026,-100.1\n"
        "B,09:28:09 AM 04/20/2026,150.1\n"
    )
    observations, errors = read_observations(filename)
    self.assertEqual(observations, {})
    self.assertEqual(errors, [(1, "Invalid temperature"), (2, "Invalid
temperature")])

```

```

def test_calculates_statistics_and_outliers(self):
    observations = {
        "A": [
            (datetime(2026, 4, 19, 9, 28, 9), 10.0),
            (datetime(2026, 4, 20, 9, 28, 9), 20.0),
        ],
        "B": [(datetime(2026, 4, 20, 9, 28, 9), -5.0)],
    }
    self.assertEqual(station_statistics(observations), {"A": (10.0, 20.0,
15.0), "B": (-5.0, -5.0, -5.0)})
    self.assertEqual(station_outliers(observations)["A"][1:], (20.0, 15.0))

def test_writes_sorted_statistics_with_one_decimal(self):
    filename = self.create_input_file("")
    write_statistics(filename, {"Zoo": (1, 4, 2.5), "Alpha": (-2, 3, 0.5)})
    with open(filename, "r", encoding="utf-8") as output_file:
        self.assertEqual(output_file.read(), "Alpha,-
2.0,3.0,0.5\nZoo,1.0,4.0,2.5\n")

def test_missing_file_raises_file_not_found_error(self):
    with self.assertRaises(FileNotFoundError):
        read_observations("missing-temperature-file.csv")

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] in {"-m", "--test"}:
        sys.argv.pop(1)
        unittest.main()
    else:
        main()

```

